

VMC AI Call Center – Master Plan

AI powered inbound call system to accept citizen complaints, extract structured complaint data, confirm details with the caller, file cases, and present a live admin dashboard for VMC officials.

Table of Contents

- [Project Summary](#)
 - [Goals and Success Metrics](#)
 - [Scope and Out of Scope](#)
 - [High Level Architecture](#)
 - [Component Breakdown](#)
 - [Data and Message Schemas](#)
 - [Conversation Flow and State Machine](#)
 - [Prompts and System Prompt Examples](#)
 - [Backend Design](#)
 - [Frontend Design](#)
 - [Security, Privacy and Compliance](#)
 - [Observability and Monitoring](#)
 - [Testing Plan](#)
 - [Deployment and Infrastructure](#)
 - [Risks and Mitigations](#)
 - [Roadmap and Milestones](#)
 - [Appendix: Example Code Snippets](#)
-

Project Summary

This project builds an AI powered inbound call system for Vadodara Municipal Corporation (VMC). People call a helpline. The system converses in Hindi, Gujarati, and English using STT, a structured text LLM for reasoning and extraction, and TTS for confirmations. Complaints are auto filed into a backend and surfaced on a real time admin dashboard built with React + Vite + shadcn. Twilio handles inbound telephony. The orchestrator is a Python FastAPI service that coordinates audio streaming, LLM calls, form management, and websocket events.

Goals and Success Metrics

Primary Goals

- Accept inbound calls 24x7 and extract complaints reliably
- Auto-fill complaint forms with > 85% accuracy for required fields
- Provide live dashboard for officials with recent call logs and statuses
- Allow human escalation and takeover
- Respect privacy and comply with local laws

Success Metrics

- Calls answered time < 5 seconds
 - Automated form fill accuracy > 85% on required fields
 - Escalation rate < 10% for normal complaints
 - Average call handling time < 4 minutes
 - System uptime > 99%
-

Scope and Out of Scope

In Scope

- Inbound PSTN calls via Twilio
- STT via OpenAI Whisper
- Gemini 3.0 flash for structured extraction and reasoning
- Sarvam AI for TTS
- FastAPI orchestrator that routes audio, events, and data
- Reactive admin dashboard with live events and recent call logs
- DB schema and basic reporting
- Handoff to human operator via dashboard

Out of Scope for Hackathon MVP

- Native Gemini Live audio streaming integration
 - Full workforce management, agent scheduling and advanced CRM features
 - Deep voice cloning or voice personalization for citizens
 - Integration with multiple municipal systems beyond basic logging and alerts
 - Offline deployments for complete on-premise privacy
-

High Level Architecture

Caller (PSTN)

↓ (SIP/WebRTC)

Twilio Telephony

↓ (Audio stream)

FastAPI Orchestrator

↳ OpenAI Whisper STT (chunked audio) → transcripts

↳ Gemini 3.0 Flash LLM → structured JSON

↳ Sarvam AI TTS ← text

↳ Postgres DB

↳ React + Vite + shadcn Dashboard (WebSocket events)

Short Description

- Twilio receives inbound call and streams audio to the FastAPI orchestrator
 - Orchestrator forwards audio to Whisper in manageable chunks and receives transcripts
 - Orchestrator batches stable transcripts and sends to Gemini 3.0 flash, with a strict system prompt requiring structured JSON for complaint forms and next actions
 - Gemini responds with structured JSON including extracted fields, missing fields, and next action
 - Orchestrator uses TTS (Sarovam AI) to speak confirmations and follow ups, and logs structured data to Postgres
 - Dashboard receives websocket events to show live calls, call detail and case status
-

Component Breakdown

Telephony - Twilio

Responsibilities:

- Receive inbound PSTN calls to a VMC helpline
- Provide media streaming (WebRTC or Twilio Media Streams) to the FastAPI orchestrator
- Play hold prompts and TTS audio returned by Sarvam through Twilio
- Accept DTMF or language selection input for initial IVR

Key Twilio Features:

- Twilio Programmable Voice with Media Streams or Voice Webhooks
- TwiML to play hold or prompt messages
- Twilio SIP if required to connect to other PBX

Notes:

- Use Twilio WebRTC streaming to send real time audio chunks to orchestrator
- Use a short IVR at start: "For Hindi press 1. For Gujarati press 2. For English press 3."

Speech-to-Text - OpenAI Whisper

Responsibilities:

- Convert inbound streaming audio to text
- Return partial and final transcripts
- Support Hindi, Gujarati, English with reasonable accuracy

Approach:

- Use Whisper API for near real time transcription
- Use VAD (voice activity detection) in orchestrator to detect turn ends
- Send partial transcripts for dashboard live view, and final transcript to Gemini

Notes:

- Whisper can be used in streaming or batch mode depending on latency and cost
- Keep a short buffering window and use silence detection to decide when to commit a transcript to Gemini

LLM Reasoning and Structured Output - Gemini 3.0 Flash

Responsibilities:

- Extract structured complaint data from transcripts
- Decide next action (ask missing field, confirm, file complaint, escalate)
- Return only strict JSON payloads when invoked for structured outputs

Key Considerations:

- Use Gemini 3.0 flash with a system prompt that forces JSON-only output
- Provide conversation history, current form state and confidence thresholds in each request
- Have a fallback path if model outputs text instead of valid JSON (server-side validation)

Example Capabilities:

- Map natural language into category enum (garbage_missed, water_shortage, streetlight)
- Validate addresses and transform to canonical form when possible
- Indicate confidence per field

Text-to-Speech - Sarvam AI

Responsibilities:

- Create natural voice confirmations and follow ups for callers
- Support multilingual TTS (Hindi, Gujarati, English)
- Low latency playback via Twilio

Integration:

- Orchestrator sends "speak" text to Sarvam AI and receives audio file or stream
- Orchestrator instructs Twilio to play this audio segment to the caller

Notes:

- Cache common phrases and confirmations in audio to reduce latency
- Consider short pre-recorded prompts for IVR and fallback messages

Orchestrator - FastAPI Python

Responsibilities:

- Coordinate the whole flow
- Accept audio streams from Twilio
- Call Whisper for STT
- Call Gemini for structured extraction
- Call Sarvam for TTS
- Manage call state machine
- Store records in Postgres
- Broadcast websocket events to dashboard
- Maintain short-term session store (Redis) for active calls
- Provide REST APIs for dashboard operations, human takeover, and admin functions

Non-Functional Responsibilities:

- Rate limit and concurrency control
- Graceful retry and backoff when external APIs fail
- Secure authentication and privileges
- Logging, tracing, metrics

Database and Storage

- **Primary DB:** PostgreSQL
- **Tables:** calls, complaints, complaint_fields, transcripts, events, users, audit_logs

- **Object Storage:** S3 (or S3-compatible) for audio recordings
- **Cache:** Redis for ephemeral call session state and locks

Frontend - React + Vite + shadcn

Features:

- Dashboard home with KPIs
- Live recent call logs table with status badges
- Call detail view showing transcript timeline, structured fields and audio playback
- Search and filter complaints
- Human takeover controls (whisper, barge)
- Settings for language text, thresholds and escalation rules

UX Notes:

- Use WebSocket to receive `partial_transcript` and `form_update` events
 - Provide confirmation and action buttons: Assign, Escalate, Close, Replay Recording
 - Show confidence bars for each extracted field
-

Data and Message Schemas

Call Session Object (Example)

```
{
  "session_id": "call_20250715_00001",
  "caller": "+9198210XXXXX",
  "language": "hi",
  "start_ts": "2025-07-21T09:52:30+05:30",
  "status": "active",
  "current_state": "collecting",
  "last_activity_ts": "2025-07-21T09:53:10+05:30",
  "metadata": {
    "twilio_call_sid": "CAXXX",
    "twilio_connection": "...",
    "recording_url": null
  }
}
```

Structured Complaint JSON Schema

```

{
  "$id": "https://vmc.example/schemas/complaint.json",
  "type": "object",
  "properties": {
    "complaint_type": {
      "type": "string",
      "enum": ["missed_collection", "overflow", "illegal_dumping", "street"]
    },
    "description": {"type": "string"},
    "address": {"type": "string"},
    "locality": {"type": "string"},
    "pincode": {"type": "string"},
    "contact_number": {"type": "string"},
    "reported_date": {"type": "string", "format": "date-time"},
    "urgency": {"type": "string", "enum": ["low", "medium", "high"]},
    "attachments": {
      "type": "array",
      "items": {"type": "string", "format": "uri"}
    }
  },
  "required": ["complaint_type", "address", "contact_number"]
}

```

Websocket Event Types

All websocket events include `session_id`, `timestamp` and `type`.

- **call_started** - { `session_id`, `caller`, `start_ts` }
- **partial_transcript** - { `session_id`, `text`, `language`, `confidence`, `is_final` }
- **final_transcript** - { `session_id`, `text`, `confidence` }
- **form_update** - { `session_id`, `form`: { `...fields` }, `changes`: [{ `field`, `from`, `to`, `confidence` }] }
- **speak_action** - { `session_id`, `tts_text`, `tts_audio_url` }
- **case_created** - { `session_id`, `case_id`, `status` }
- **call_ended** - { `session_id`, `end_ts`, `duration_sec` }
- **escalation_alert** - { `session_id`, `reason`, `priority` }

API Endpoints (High Level)

- `POST /api/calls/start` - Twilio webhook to indicate new incoming call

- `POST /api/calls/{session_id}/audio` - streaming or chunked audio endpoint
 - `POST /api/calls/{session_id}/dtmf` - DTMF events
 - `GET /api/calls/{session_id}` - call status and details
 - `POST /api/complaints` - create complaint manually (dashboard)
 - `GET /api/complaints` - list / filter complaints
 - `POST /api/admin/takeover/{session_id}` - human take over control
 - `GET /api/stats` - admin dashboards
-

Conversation Flow and State Machine

Use a small deterministic state machine per call session.

States

- **INIT** - Initial state after call pickup. Play language IVR
- **LISTENING** - Receiving caller speech, sending to STT
- **PROCESSING** - Waiting for Gemini structured output
- **ASKING** - Session speaking a follow up question and listening for reply
- **CONFIRMING** - Reading back extracted fields and waiting for confirmation
- **FILING** - Creating complaint in DB
- **ENDED** - Call ended

Transitions

- INIT → LISTENING when IVR choice received
- LISTENING → PROCESSING on VAD triggered final transcript or turn boundary
- PROCESSING → ASKING if missing fields exist and confidence below threshold
- PROCESSING → CONFIRMING if fields extracted and confidence acceptable
- CONFIRMING → FILING on user confirmation
- FILING → ENDED after complaint created and confirmation spoken
- Any state → ENDED if caller hangs up or timeout exceeded
- Any state → ESCALATED if safety keyword or low confidence for core fields

Decision Rules

- Field accepted if confidence ≥ 0.8 , otherwise flagged as low confidence and included in `missing_fields` list
- After 2 cycles of clarification for the same field without resolution, escalate to human
- If safety keywords or legal risks detected, escalate to human and log the call for audit

Prompts and System Prompt Examples

System Prompt Template for Gemini 3.0 Flash

You are a structured complaint extractor for the Vadodara Municipal Corporation.

You will receive as input a JSON object with fields:

- "session_id": string
- "history": array of {role: "user"|"assistant", "text": string}
- "current_form": object with fields that have already been filled

Your output must be only a valid JSON with this schema:

```
{
  "intent": "complaint"|"other",
  "complaint_type": "...",
  "fields": {...},
  "missing_fields": ["address", "pincode"],
  "confidence": {"address": 0.87, ...},
  "next_action": "ask"|"confirm"|"file"|"escalate",
  "speak": "The exact short sentence you want the TTS to speak to the caller."
}
```

Do not output any text outside the JSON. If uncertain about any required field, put it in missing_fields and set next_action to "ask". Always keep "speak" actionable and short, less than 20 words. Use address normalization when possible.

Example Conversation Payload

```
{
  "session_id": "call_1234",
  "history": [
    {"role": "user", "text": "Mera kachra kal nahi uthaaya gaya, 12 MG Road."},
  ],
  "current_form": {}
}
```

Expected Gemini Output

```
{
  "intent": "complaint",
  "complaint_type": "missed_collection",
  "fields": {
    "address": "12 MG Road, Sayajiganj, Vadodara",
    "contact_number": "+9198210XXXXX"
  },
  "missing_fields": ["pincode"],
  "confidence": {"address": 0.92, "contact_number": 0.85},
  "next_action": "ask",
  "speak": "Can you tell me the pincode for your address?"
}
```

Clarification Prompts Pattern

- If next_action is **ask**, Orchestrator should call TTS with speak then go to ASKING state
 - If next_action is **confirm**, orchestrator should read back key fields: "I have missed trash pickup at 12 MG Road. Is this correct?"
 - If next_action is **file**, orchestrator should file and reply: "Your complaint has been registered. Your ID is VMC-2025-00007."
-

Backend Design

Project Layout (Suggested)

```
/backend
  /app
    /api
      calls.py
      complaints.py
      admin.py
    /services
      twilio_client.py
      stt_client.py
      tts_client.py
      gemini_client.py
```

```
state_machine.py
websocket_broadcaster.py
/models
  call.py
  complaint.py
  user.py
/db
  migrations/
/schemas
  complaint_schema.json
main.py
/tests
Dockerfile
requirements.txt
```

Key Modules

- **twilio_client.py**
 - Accept incoming event, translate Twilio media stream to service audio chunks
 - Control TwiML to play audio returned by TTS
- **stt_client.py**
 - Buffer audio, detect VAD boundaries, call Whisper API for partial and final transcripts
- **gemini_client.py**
 - Send structured prompt with history and current_form, parse JSON response, validate against JSON schema
- **tts_client.py**
 - Send speak text to Sarvam AI, receive audio file/stream, upload to S3 if needed, return URL for Twilio
- **state_machine.py**
 - Implement deterministic state transitions and decision rules
- **websocket_broadcaster.py**
 - Broadcast events to connected dashboard clients
- **Database models**

- Calls, complaints, transcripts, audit logs

Streaming and Async Considerations

- Use async endpoints in FastAPI for audio streaming
- Use background workers (Celery, RQ, or asyncio tasks) for heavy tasks like Gemini calls
- Keep short synchronous loops for low latency: send transcript to Gemini only at turn boundaries
- Use Redis to store session state and locks to avoid concurrency issues for a session

Resilience

- Implement retry with exponential backoff for external API calls
 - Circuit breaker for LLM and TTS to avoid cascading failures
 - Graceful degradation: if Gemini is down, fallback to limited rule-based extraction
 - Persist audio recordings so reprocessing is possible
-

Frontend Design

Pages

- **Dashboard**
 - KPIs, call volume, automated logging rate
 - List of Recent Call Logs (live)
 - Escalated Complaints snapshot
- **Call Detail Page**
 - Timeline with transcripts (user/AI)
 - Extracted structured fields and confidence bars
 - Audio playback for call recording
 - Action buttons: Assign, Escalate, Replay, Takeover
- **Complaints List and Filters**
 - Search by case id, phone, location, type
- **Settings**
 - Thresholds, language configuration, escalation rules, admin users

Components

- KPI Card
- Live Table for call logs with streaming updates
- Transcript timeline component with partials and final transcripts
- Form display component for structured fields with inline edit
- Notification panel for alerts
- Modal for human takeover controls

Typescript Interfaces

```
export interface CallLog {
  sessionId: string;
  caller: string;
  location?: string;
  calledAt: string;
  durationSeconds?: number;
  status: "processing"|"registered"|"escalated"|"failed"|"pending";
}

export interface FormUpdateEvent {
  sessionId: string;
  form: Record<string, any>;
  changes: Array<{ field: string; from: any; to: any; confidence?: number
  timestamp: string;
}
```

UX Notes

- Show confidence indicators next to each extracted field
 - Enable in-place correction by operators which will update the complaint record
 - Highlight urgent records and repeated complaints
-

Security, Privacy and Compliance

- Announce call recording and data use at call start and store consent flag
- Encrypt audio and transcripts in transit using TLS
- Encrypt PII at rest (e.g., contact numbers)
- Mask personal data in dashboard views, show full data only to authorized roles
- Role based access control for dashboard
- Retention policy: delete recordings after defined retention period unless flagged

- Audit logs for all automated actions and operator interventions
 - GDPR/India privacy law compliance: provide deletion mechanism and data export for citizens on request
-

Observability and Monitoring

Metrics to Collect

- Calls per minute, average call duration
- Gemini latency and error rates
- Whisper latency and transcription error rate
- TTS latency
- Form extraction confidence distribution
- Escalation rates and reasons

Logs

- Structured logs with correlation id / session_id for traceability

Tracing

- Use OpenTelemetry to trace requests across FastAPI → Whisper → Gemini → Sarvam

Alerts

- Alert ops on high error rates, LLM 5xx errors, elevated drop in automated logging rate

Dashboards

- Grafana with charts for the metrics above
-

Testing Plan

- **Unit tests** for all modules
- **Integration tests**
 - Simulate Twilio webhook with recorded audio and assert final complaint creation
 - Mock Gemini responses to test various scenario branches
- **End to end tests**

- Play sample audio for each complaint type and assert DB records
 - **Load testing**
 - Use k6 to simulate concurrent calls and check orchestration throughput
 - **Human audit**
 - Random sampling of created complaints and manual verification of accuracy
-

Deployment and Infrastructure

Suggested Cloud Setup

- Kubernetes cluster (EKS/GKE/AKS) or Docker Compose for MVP
- FastAPI deployed as container behind an ingress
- Postgres managed service (RDS/CloudSQL)
- Redis managed instance for session store
- S3 for audio storage
- CI/CD pipeline using GitHub Actions

Components to Run as Services

- FastAPI app (autoscale)
- Background worker for heavy LLM tasks (Celery or asyncio worker)
- Frontend static hosting on Vite with Cloudflare or S3 + CDN

Secrets and Credentials

- Store API keys in a secret manager
- Rotate keys periodically
- Use per-service least privilege policies

Cost Considerations

- Whisper and Gemini costs will be the main variable cost
 - TTS costs depend on audio length; cache repeat confirmations
 - Twilio costs for calls and audio streams
 - Use rate limiting and batching to control LLM usage
-

Risks and Mitigations

Risk	Mitigation
Poor STT quality for local accents	Add language/region selection IVR, sample training utterances for Whisper finetuning, show transcript for operator correction
Model hallucination or incorrect structured output	Use strict system prompts, validate JSON server side, require confirmation for critical fields
Network or API downtime	Fallback rules, retry policies and human takeover button
Privacy concerns or regulatory issues	Explicit consent, data retention policy, encryption and RBAC
High latency diminishes user experience	Shorter prompts, partial responses for immediate feedback, asynchronous processing

Roadmap and Milestones

Target for a 2 week hackathon sprint and follow up improvements.

Week 0 – Prep

- Finalize stack and accounts for Twilio, OpenAI, Gemini, Sarvam
- Create project skeleton and repository

Week 1 – Core MVP

- Day 1: Twilio inbound call + media stream reach orchestrator
- Day 2: Whisper integration for STT with VAD
- Day 3: Gemini adapter with structured prompt and test harness
- Day 4: Basic TTS integration and Twilio playback
- Day 5: Store structured data in Postgres and create basic dashboard
- Day 6-7: Polish, demo flow, fix bugs

Week 2 – Enhancements and Reliability

- Implement websocket live events to dashboard
- Implement state machine and retry logic
- Add admin takeover controls and human in the loop
- Add logging, metrics and basic alerting
- Run end to end testing and load tests

Post Hackathon

- Optimize LLM prompts and confidence thresholds
 - Add worker queueing, autoscaling and cost optimizations
 - Add deeper analytics and reporting
 - Extend integration with municipal systems
-

Appendix: Example Code Snippets

1. Simple FastAPI Endpoint to Start a Call (Twilio Webhook)

```
from fastapi import FastAPI, Request
from pydantic import BaseModel

app = FastAPI()

@app.post("/api/calls/start")
async def call_start(req: Request):
    data = await req.form()
    call_sid = data.get("CallSid")
    from_number = data.get("From")
    session_id = f"call_{call_sid}"
    # persist call session to db, create session in redis
    # respond with TwiML to accept audio
    return "<Response><Start><Stream url='wss://your-orchestrator.example.com'></Stream></Start></Response>"
```

2. Example Gemini Call Payload

```
def build_gemini_payload(session_id, history, current_form):
    system_prompt = open("prompts/gemini_system.txt").read()
    return {
        "model": "gemini-3.0-flash",
        "system": system_prompt,
        "input": {
            "session_id": session_id,
            "history": history,
            "current_form": current_form
        },
        "response_format": {
```

```
        "type": "json_schema",
        "schema": load_json("schemas/complaint_schema.json")
    }
}
```

3. Websocket Event Example (Server to Dashboard)

```
{
  "type": "form_update",
  "session_id": "call_20250715_00001",
  "timestamp": "2025-07-21T09:53:25+05:30",
  "form": {
    "complaint_type": "missed_collection",
    "address": "12 MG Road, Sayajiganj"
  },
  "changes": [
    { "field": "address", "from": null, "to": "12 MG Road, Sayajiganj", "confidence": 0.9 }
  ]
}
```

4. JSON Schema for Gemini Response

```
{
  "type": "object",
  "properties": {
    "intent": { "type": "string" },
    "complaint_type": { "type": "string" },
    "fields": { "type": "object" },
    "missing_fields": { "type": "array", "items": { "type": "string" } },
    "confidence": { "type": "object" },
    "next_action": { "type": "string" },
    "speak": { "type": "string" }
  },
  "required": [ "intent", "fields", "next_action", "speak" ]
}
```

Final Summary

This document describes a full plan to build an AI driven municipal call center for VMC using:

- **Gemini 3.0 Flash** for text reasoning and structured output
 - **OpenAI Whisper** for speech recognition
 - **Sarvam AI** for natural TTS
 - **FastAPI** for orchestration and business logic
 - **PostgreSQL and Redis** for persistent and ephemeral storage
 - **Twilio** for telephony
 - **React + Vite + shadcn** for the admin dashboard
-

Document Version: 1.0

Last Updated: January 2026

Status: Ready for Implementation